

Servicio de eventos de guía

Ingesta de eventos de estado de una guía y publicación a una cola de mensajes. El diseño gira alrededor de tres hechos del dominio: la red falla, los clientes reintentan y el estado de una guía tiene reglas.

Cubre el **Eje 1 · Arquitectura de la solución** del enunciado. El ejercicio previo está en el repositorio.

CANDIDATO Mario Madera	REPOSITORIO github.com/mariod90/servicio-eventos-guia-tcc	STACK Node.js 22 · TypeScript · Express · SQS	ALCANCE Ruta de ingreso, acotada a 2 horas
--	---	---	--

Alto volumen, sin perder eventos

El enunciado lo plantea así: en temporadas pico el volumen de eventos se dispara, y hay que capturarlos, actualizar el estado casi en tiempo real y notificar al cliente sin perder ninguno ni degradar la operación. Cada exigencia deja una marca concreta en el diseño.

alta concurrencia

Aceptar rápido, procesar aparte

Los eventos llegan de varias plataformas a la vez y el pico multiplica el volumen. El servicio valida y encola; quien notifica al cliente trabaja a su ritmo.

sin perder eventos

Idempotencia obligatoria

Un *timeout* no significa que el evento no se procesó. El cliente reintenta, y sin clave de idempotencia cada reintento sería una notificación de más.

casi en tiempo real

El orden de llegada no es el de los hechos

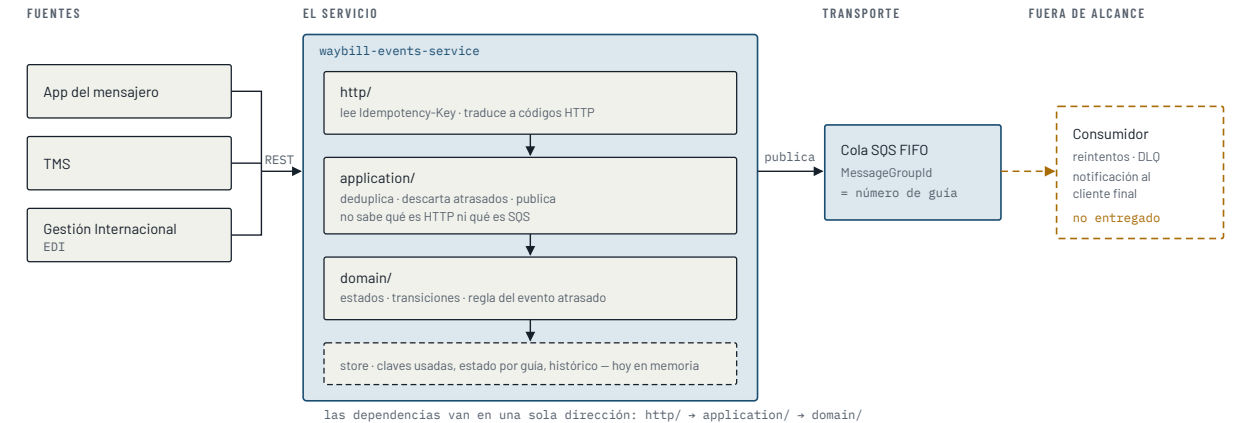
El mensajero pierde cobertura y sincroniza una hora después. El estado se decide con la hora en que ocurrió el evento, no con la de llegada.

equipo pequeño

Que se pueda cambiar sin miedo

Las reglas de negocio viven separadas del transporte HTTP y de la cola, y se prueban sin levantar infraestructura: catorce pruebas en menos de un segundo.

De la fuente a la notificación



El servicio cubre la ruta de ingreso completa —de la petición REST al mensaje en la cola—. Lo dibujado en ámbar existe en el diseño pero no en el entregable.

Qué le pasa a un evento, en orden

01	¿Ya vi antes esta Idempotency-Key ? La manda el cliente y es la misma en todos sus reintentos.	sí → 200 DUPLICATE no publica de nuevo
02	¿El cuerpo es válido? Estado conocido, fecha ISO con zona horaria, contador presente.	no → 400 INVALID_BODY
03	¿Ocurrió después del último estado que tengo? Comparo occurredAt , y desempato con sequence .	no → 202 DISCARDED_AS_STALE se guarda, no cambia el estado
04	¿La transición está permitida? Máquina de estados explícita, seis estados.	no → 422 INVALID_TRANSITION devuelve origen y destino
05	Reservo la clave y publico a la cola. Si la publicación falla, libero la clave para que el reintento si entre.	falla → 502 PUBLISH_FAILED
06	Registro el evento y actualizo el estado de la guía.	202 ACCEPTED

EL ORDEN NO ES CASUAL

Deduplico *antes* de validar, y descarto atrasados *antes* de comprobar la transición: al revés, un evento viejo que llega tarde saldría rechazado por "transición inválida" cuando esa transición nunca ocurrió.

Idempotencia, orden y reintentos

Catorce pruebas con el runner que trae Node, sin librerías externas. Estas cuatro son las que sostienen el diseño.

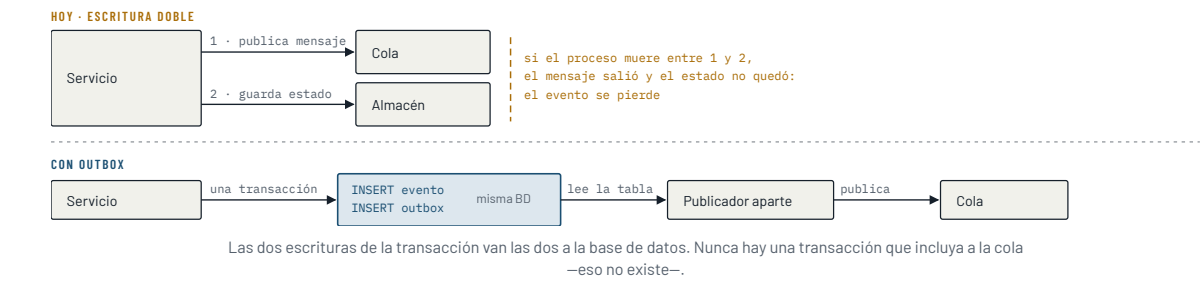
RIESGO	CÓMO LO ATACO	PRUEBA QUE LO CUBRE
Evento duplicado	Clave de idempotencia obligatoria, enviada por el cliente. La reservo <i>antes</i> de publicar, así dos peticiones simultáneas con la misma clave no publican las dos.	La misma clave dos veces devuelve <code>DUPLICATE</code> y publica una sola vez
Evento atrasado o desordenado	Comparo por la hora en que ocurrió el hecho, no por la de llegada. Y si la cola es FIFO, el <code>MessageGroupId</code> es el número de guía: orden garantizado por guía, paralelismo entre guías distintas.	Un evento atrasado se guarda en el histórico pero no cambia el estado actual
Estado imposible	Máquina de estados explícita en el dominio. <code>ENTREGADA</code> y <code>DEVUELTA</code> son terminales: no salen a ningún otro estado.	Una transición inválida devuelve <code>422</code> con el estado de origen y el de destino
La cola está caída	Libero la clave reservada y respondo <code>502</code> . El cliente reintenta con la misma clave y esta vez sí entra, en lugar de recibir un "duplicado" que no lo era.	Si falla la publicación, la clave queda libre para el reintento

Lo que decidí, y lo que cuesta

DECISIÓN	POR QUÉ	LO QUE CUESTA
La clave de idempotencia la manda el cliente	Si la calculara yo con un hash del cuerpo, un reintento con distinta hora de recepción daría otra clave y no serviría de nada.	Dependo de que el cliente la mande bien. Por eso es obligatoria y respondo <code>400</code> si falta.
Un duplicado responde <code>200</code> , no <code>409</code>	El cliente reintentó, no se equivocó. Un error lo empujaría a reintentar otra vez.	Hay que leer el campo <code>result</code> del cuerpo para distinguirlo de un alta real.
Tres capas, no una ni cinco	El escenario dice que los eventos llegan del TMS y de Gestión Internacional. El día que una fuente entre por cola en vez de REST, el caso de uso se reutiliza tal cual.	Un salto más al leer el código. Por eso la consulta de estado <i>no</i> tiene capa: hoy no tiene reglas.
Cola FIFO	Garantiza el orden dentro de una misma guía sin que el consumidor tenga que reordenar.	Menos rendimiento que la estándar. Si molestara, usaría estándar y ordenaría por <code>occurredAt</code> .
TypeScript sin paso de compilación	El resultado de validar es un tipo unión, así que el compilador me obliga a tratar el caso de error antes de poder tocar el evento. Y Node ya ejecuta <code>.ts</code> directamente.	Sube el mínimo de Node a la 22.18.

El problema que sé que tengo

Hoy escribo en dos sitios que no comparten transacción: publico en la cola y después registro el estado. Si el proceso se cae entre esas dos líneas, el mensaje ya salió pero el estado no quedó —y como la clave está reservada, el reintento del cliente responde `DUPLICATE`—.



LO QUE EL OUTBOX RESUELVE, Y LO QUE NO

Convierte “se puede perder” en “se puede duplicar”: si el publicador se cae después de enviar y antes de marcar la fila, reenvía. Por eso el consumidor tiene que ser idempotente. **No lo implementé**: sin base de datos no tenía sentido, y no lo he usado en producción. Prefiero señalarlo yo antes que entregarlo a medias.

Escalabilidad, observabilidad y seguridad

Separo lo que ya está resuelto en el código de lo que hoy falta.

escalabilidad

La cola absorbe el pico

Si el consumidor se retrasa, la ingesta no se degrada: se acumula trabajo en la cola, no peticiones en espera.

Cuando el estado salga de memoria a base de datos, el servicio queda sin estado y admite varias réplicas detrás del balanceador, escalando por CPU o por profundidad de la cola.

El orden dentro de una guía lo garantiza el `MessageGroupId`, no el número de instancias.

observabilidad

Rastrear una guía completa

Hay log estructurado en JSON con el número de guía y el id del evento como campos, no dentro del texto: se sigue el recorrido entero de una guía filtrando por un campo.

Falta: contadores de eventos aceptados, duplicados, descartados y fallidos; y alerta sobre la profundidad de la cola y la antigüedad del mensaje más viejo, que es la señal temprana de que el consumidor no da abasto.

seguridad

Menos superficie que proteger

Hoy no hay autenticación. Entre servicios iría mTLS o un token OAuth2 con alcance por operación.

Lo que sí es una decisión tomada: el mensaje lleva el número de guía, no los datos del destinatario. Quien necesite nombre o dirección los consulta al servicio que corresponda.

Menos datos personales viajando, y menos exposición frente a la Ley 1581.

Integración continua y despliegue

ETAPA	QUÉ PASA, Y POR QUÉ ASÍ
Integración continua	En cada <i>pull request</i> : instalación limpia, <code>npm run typecheck</code> y <code>npm test</code> . Las dos puertas ya existen en el repositorio, así que no hay que inventar el pipeline sino conectarlo. Si alguna falla, el cambio no se integra.
Empaquetado	Imagen de contenedor con la versión de Node fijada. Al no haber paso de compilación, la imagen es el código y sus dependencias: lo que se prueba es lo que se despliega.
Configuración	Por variables de entorno (<code>SQS_QUEUE_URL</code> , <code>AWS_REGION</code>), nunca en el repositorio. El mismo artefacto sirve para todos los entornos; lo único que cambia es a qué cola apunta.
Despliegue	Sustitución gradual de instancias detrás del balanceador. <code>GET /health</code> ya existe y es la sonda: si una instancia nueva no responde, no recibe tráfico y se vuelve a la versión anterior.

EL ORDEN IMPORTA MÁS QUE LA HERRAMIENTA

Antes del pipeline haría la base de datos y el outbox de la lámina 07. Sin eso, la entrega continua solo consigue desplegar más rápido un servicio que todavía puede perder un evento.

Dónde puse el límite, y por qué

El ejercicio es opcional y con tope de dos horas, así que lo acoté a la ruta de ingreso. Preferí **una ruta completa y probada** antes que un sistema entero a medias: el consumidor, la base de datos y el despliegue no son olvidos, son recortes con fecha.

Elegí **Node con TypeScript** porque es donde tengo más profundidad hoy y quería poder defender cada línea de este código. La prueba menciona Java con Spring Boot y Angular como tecnologías principales, y también servicios en Node.

Sobre la cola: usé **SQS** porque es lo que he tocado en AWS. **No he trabajado con Kafka ni con RabbitMQ en producción**. La diferencia que sí tengo clara para un caso como este es que Kafka conserva el log y permite reprocesar desde el principio, mientras que SQS borra el mensaje cuando el consumidor lo confirma —para reprocesar una guía completa, esa capacidad es un argumento fuerte a favor de Kafka—.

PARA REVISARLO

`npm install` y `npm start` lo levantan sin AWS ni base de datos: si no hay cola configurada, usa un publicador en memoria que escribe el evento en consola.
`npm test` corre las catorce pruebas y `npm run typecheck` revisa los tipos. El README explica cada decisión con más detalle.